Zen AI Model Family

Zen Coder

Agentic Code Generation Models (4B - 1T)

Technical Report v2.0

Antje Worring, Zach Kelling*
research@hanzo.ai

Zoo Labs Foundation
foundation@zoo.ngo

December 2025

Abstract

Zen Coder is a packaging and light fine-tuning of Alibaba's open-weight **Qwen Coder** models—**Qwen2.5-Coder** [1] for the dense tiers and **Qwen3-Coder** [2] for the Mixture-of-Experts tiers—redistributed under the Zen brand for agentic coding workflows. It is *not* a from-scratch model and does not introduce a new architecture or pretraining corpus; the upstream weights, tokenizer, and architecture are Qwen's, the pretraining (5.5T+ code tokens for Qwen2.5-Coder) was done by the Qwen team, and we contribute only deployment packaging and optional task-specific fine-tuning. The Apache-2.0–licensed Qwen Coder checkpoints are redistributed under Apache-2.0 with attribution to Alibaba. This report documents which upstream checkpoint backs each Zen Coder tier, the fine-tuning configuration we apply, and the intended deployment surface.

Contents

1 Introduction **3**
1.1 Model Family . . . 3
1.2 What Zen Coder Adds (and Does Not) . . . 3

2 Upstream Training and Data **3**

3 Model Architectures **4**
3.1 Dense Tiers (Qwen2.5-Coder) . . . 4
3.2 MoE Tier (Qwen3-Coder) . . . 4

4 Fine-Tuning Methodology **4**
4.1 Fine-tuning Configuration . . . 4

5 Usage **5**
5.1 Installation . . . 5
5.2 Training Example . . . 5
5.3 Inference Example . . . 5

6 Relationship to Upstream **6**

*zach@lux.network

7 Evaluation **6**

8 Licensing & Access **6**
8.1 Models 6
8.2 Fine-tuning Data 6

9 Supported Organizations **7**

10 Conclusion **7**

1 Introduction

The emergence of agentic AI systems—where models interact with tools, execute multi-step plans, and maintain context across complex workflows—has created demand for capable open-weight code models. The Qwen team at Alibaba released two such families under permissive terms: **Qwen2.5-Coder** [1], a set of dense code models (0.5B–32B) continued-pretrained on over 5.5 trillion code tokens, and **Qwen3-Coder** [2], a Mixture-of-Experts agentic-coding model (480B total / 35B active). **Zen Coder** is a redistribution of these upstream checkpoints under the Zen brand, with optional task-specific fine-tuning. We make no claim to having trained these models from scratch; all pretraining, the architecture, and the tokenizer are Qwen's.

1.1 Model Family

Each Zen Coder tier is backed directly by a published Qwen Coder checkpoint, as shown below. "Base" names the exact upstream model; "status" indicates whether we currently ship a fine-tuned Zen variant or redistribute the upstream weights unchanged.

Zen tier	Size	Upstream base	VRAM	Context	Status
Zen Coder 3B	3B	Qwen2.5-Coder-3B-Instruct	8 GB	32K	Fine-tuned
Zen Coder 7B	7B	Qwen2.5-Coder-7B-Instruct	16 GB	128K	Fine-tuned
Zen Coder 14B	14B	Qwen2.5-Coder-14B-Instruct	32 GB	128K	Fine-tuned
Zen Coder 32B	32B	Qwen2.5-Coder-32B-Instruct	64 GB	128K	Fine-tuned
Zen Coder MoE	480B (A35B)	Qwen3-Coder-480B-A35B-Instruct	8×H100	256K	Repackaged

Table 1: Zen Coder tiers and their upstream Qwen Coder bases. Qwen3-Coder supports 256K context natively and up to 1M via extrapolation [2].

1.2 What Zen Coder Adds (and Does Not)

- **Packaging**: distribution under the `zenlm` namespace with Zen deployment tooling and serving defaults; the weights remain Qwen's.
- **Optional fine-tuning**: light LoRA fine-tuning on task-specific data for some tiers (Section 4); the MoE tier is redistributed unchanged.
- **Attribution**: Apache-2.0 upstream checkpoints are redistributed under Apache-2.0 with attribution to Alibaba/Qwen; no new pretraining corpus is introduced.

Not claimed. Zen Coder does not introduce a novel architecture, a from-scratch pretraining run, or a proprietary training dataset. Earlier drafts of this report described a "Zen Agentic Dataset" of billions of tokens of real Claude Code sessions and a "Zen MoDE" architecture; no such dataset or architecture backs these models, and those claims have been removed.

2 Upstream Training and Data

Zen Coder introduces no pretraining corpus of its own. All pretraining was performed by the Qwen team and is documented in their technical reports; we summarize it here only for completeness and direct the reader to the upstream sources for authoritative detail.

- **Qwen2.5-Coder** (dense tiers) is built on the Qwen2.5 architecture and continued-pretrained on a corpus of more than 5.5 trillion tokens with data cleaning, synthetic data generation, and balanced data mixing, as described in the Qwen2.5-Coder Technical Report [1]. The

composition and provenance of that corpus are Qwen's; we do not have, and do not assert, any further detail.

- **Qwen3-Coder** (MoE tier) is an agentic-coding MoE trained by Qwen with long-horizon reinforcement learning on software-engineering benchmarks [2].

Any task-specific fine-tuning we apply on top of these checkpoints (Section 4) uses small, internally curated instruction sets; it is a fine-tune, not a pretraining run, and the volume is negligible relative to the upstream corpora above.

3 Model Architectures

The architectures below are inherited unchanged from the corresponding Qwen Coder upstream checkpoints; the figures are reproduced from Qwen's model cards and reports [1, 2].

3.1 Dense Tiers (Qwen2.5-Coder)

Zen tier	Upstream architecture	Layers	Hidden Dim
Zen Coder 7B	Qwen2.5-Coder-7B	28	3584
Zen Coder 14B	Qwen2.5-Coder-14B	48	5120
Zen Coder 32B	Qwen2.5-Coder-32B	64	5120

Table 2: Dense tier architectures (inherited from Qwen2.5-Coder [1]).

3.2 MoE Tier (Qwen3-Coder)

The MoE tier is redistributed from Qwen3-Coder unchanged:

Zen tier	Total Params	Active Params	Experts
Zen Coder MoE	480B	35B	160 experts, top-8

Table 3: MoE tier (inherited from Qwen3-Coder-480B-A35B [2]).

4 Fine-Tuning Methodology

We do not pretrain. The only training we perform is optional parameter-efficient fine-tuning (LoRA [3]) of the Apache-2.0 dense Qwen2.5-Coder checkpoints on small internal instruction sets, using the open-source zen-trainer wrapper around standard backends (MLX for Apple Silicon, Unsloth/DeepSpeed for NVIDIA). The Qwen3-Coder MoE tier is redistributed without fine-tuning.

4.1 Fine-tuning Configuration

Zen tier	LoRA r	LoRA α	Batch	LR
7B	64	128	4	2e-4
14B	32	64	2	1e-4
32B	16	32	1	5e-5

Table 4: LoRA fine-tuning hyperparameters for the dense tiers. The MoE tier is not fine-tuned.

Because this is a LoRA fine-tune over a small instruction set rather than a pretraining run, the compute is modest—on the order of single-GPU-hours to low tens of GPU-hours per dense tier—and we do not report large training-cost tables, which would misrepresent the scope of the work. The substantive compute cost of these models is the upstream Qwen pretraining, which we did not perform.

5 Usage

5.1 Installation

```
pip install zen-trainer
```

Listing 1: Install zen-trainer

5.2 Training Example

```
from zen_trainer import ZenTrainer

# Base is the upstream Apache-2.0 Qwen2.5-Coder checkpoint.
trainer = ZenTrainer(
    base_model="Qwen/Qwen2.5-Coder-7B-Instruct",
    dataset_path="./data/your-instruction-set.jsonl",
    output_dir="./output/zen-coder-7b",
)
trainer.train()
```

Listing 2: LoRA fine-tuning a Qwen2.5-Coder base with zen-trainer

5.3 Inference Example

```
from transformers import AutoModelForCausalLM, AutoTokenizer

model = AutoModelForCausalLM.from_pretrained(
    "zenlm/zen-coder-7b",
    torch_dtype="auto",
    device_map="auto"
)
tokenizer = AutoTokenizer.from_pretrained("zenlm/zen-coder-7b")

messages = [
    {"role": "user", "content": "Write␣a␣Python␣function␣to␣validate␣
        email␣addresses"}
]

text = tokenizer.apply_chat_template(messages, tokenize=False,
    add_generation_prompt=True)
inputs = tokenizer(text, return_tensors="pt").to(model.device)
outputs = model.generate(**inputs, max_new_tokens=512)
print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

Listing 3: Using Zen Coder for inference

6 Relationship to Upstream

Zen Coder's capabilities are, to first order, the capabilities of the underlying Qwen Coder checkpoints. Qwen2.5-Coder reports state-of-the-art results among open code models in its size class across more than ten code benchmarks [1], and Qwen3-Coder targets agentic software-engineering tasks via long-horizon reinforcement learning [2]. We refer the reader to those reports for authoritative benchmark numbers; we do not restate or modify them here, and we make no claim that Zen-brand packaging or light LoRA fine-tuning improves on the upstream scores.

What Zen Coder provides over a raw upstream download is operational: a consistent serving namespace, deployment defaults, and—for the dense tiers—optional fine-tunes to a customer's own instruction data. The value proposition is integration and convenience, not a superior base model.

7 Evaluation

We do not run a separate benchmark suite for Zen Coder; the relevant numbers are the upstream Qwen Coder results [1, 2], evaluated on standard agentic-coding benchmarks such as SWE-bench Verified, BFCL, Terminal-Bench, and LiveCodeBench. Any customer-specific fine-tune should be evaluated on the customer's own held-out tasks; we report such numbers per-engagement rather than as headline claims here.

8 Licensing & Access

8.1 Models

All Zen Coder tiers inherit the license of their upstream Qwen Coder checkpoint. The bases we use are Apache-2.0, so the redistributed Zen variants are likewise **Apache-2.0**, with attribution to Alibaba/Qwen retained in the model cards and NOTICE files:

- Zen Coder 7B / 14B / 32B: Apache 2.0 (from Qwen2.5-Coder-7B/14B/32B [1]).
- Zen Coder MoE: Apache 2.0 (from Qwen3-Coder-480B-A35B [2]).

We do not relicense these models under a proprietary "Zen Commercial License"; Apache-2.0 upstream weights are redistributed under Apache-2.0. (Note: the Qwen2.5-Coder-3B checkpoint carries the separate Qwen-Research license upstream and is therefore not offered as an Apache-2.0 Zen tier.)

8.2 Fine-tuning Data

Zen Coder ships with no proprietary pretraining dataset. The earlier "Zen Agentic Dataset" described in prior drafts does not exist as characterized and has been removed from this report. Customer fine-tunes use the customer's own data.

9 Supported Organizations

Organization	Focus	Role
Hanzo AI	AI infrastructure	Primary maintainer
Zen LM	Open model research	Model training
Zoo Labs	Decentralized AI	Research grants
Lux Network	AI compute settlement	Infrastructure

Table 5: Supporting Organizations

10 Conclusion

Zen Coder is a redistribution—and, for the dense tiers, an optional light fine-tune—of Alibaba's open-weight Qwen Coder models: Qwen2.5-Coder [1] for the dense tiers and Qwen3-Coder [2] for the MoE tier. It is not a from-scratch model, introduces no new architecture or pretraining corpus, and is offered under the upstream Apache-2.0 license with attribution to Qwen. The value of the Zen packaging is operational—consistent serving and the ability to fine-tune a strong open base to a customer's own data—rather than a claim of independent model capability.

Links

- Zen packaging: huggingface.co/zenlm
- Upstream (dense): Qwen2.5-Coder [1]
- Upstream (MoE): Qwen3-Coder-480B-A35B [2]
- Fine-tuning wrapper: github.com/zenlm/zen-trainer
- Website: zenlm.org

References

[1] B. Hui, J. Yang, et al. (Qwen Team, Alibaba). Qwen2.5-Coder Technical Report. *arXiv preprint arXiv:2409.12186*, 2024. Apache-2.0 (0.5B/1.5B/7B/14B/32B). https://github.com/QwenLM/Qwen2.5-Coder.

[2] Qwen Team, Alibaba. Qwen3-Coder: Agentic Coding in the World, 2025. Qwen3-Coder-480B-A35B-Instruct, Apache-2.0. https://github.com/QwenLM/Qwen3-Coder.

[3] E. J. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. *ICLR*, 2022.